



Lexical and Syntactic Analysis of Informal Urban Communication in Yaoundé

CS4110 — Compiler Construction

Fru Chi Ehud Neba ICTU20241812

Daniel Victor ICTU20241332

Tchelibo Ayolo Sherrylle Claire ICTU20241316

Kefeyin Hariette Sela ICTU20241471

The problem

Why ordinary parsing techniques do not simply apply

- Everyday speech in Yaoundé mixes French, English, Pidgin and Camfranglais **inside a single sentence**.
- Whitespace does not mark lexeme boundaries: Carrefour Obili is **one** place name.
- The same form changes grammatical class with context, which a lexer cannot see.

One corpus statement

Chef, drop me for Carrefour Obili.

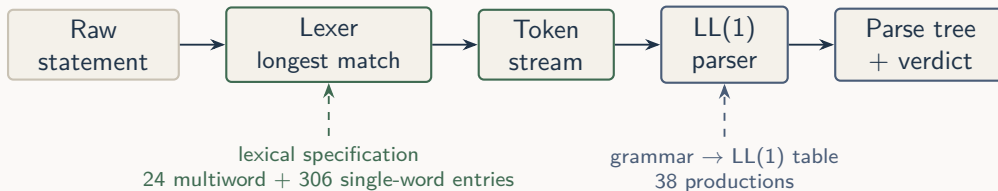
Chef — French, used as a vocative
drop — English verb, Pidgin sense
for — Pidgin preposition, not English
Carrefour Obili — one multiword
lexeme

15 of 15 collected statements draw on more than one language.

Goal Apply the **lexical** and **syntactic** phases of a compiler to this register, and report honestly where they stop working.

Method

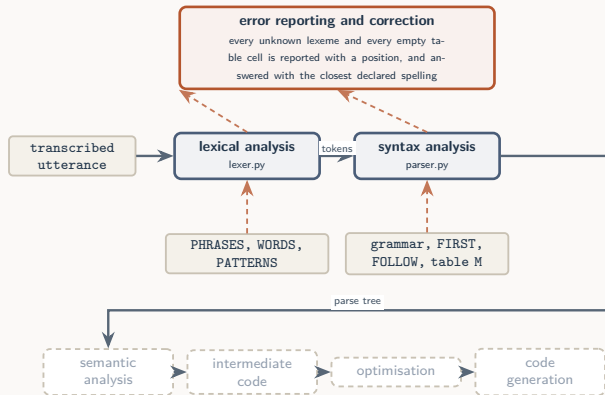
One pipeline, built and tested phase by phase



- Corpus of **15** statements, plus **6** constructed counter-examples that *must* be rejected.
- Unknown input is **reported**, never silently dropped.

Where this project sits

A compiler has six phases; we build the first two, properly



Solid: implemented, tested and measured in this project. Dashed: the synthesis phases a full compiler would add, deliberately outside the scope of a lexical and syntactic analyzer.

Analysis is implemented and measured.

Synthesis is out of scope — and saying so precisely is part of the answer.

Lexical analysis

Longest match, with multiword lexemes tried first

- **24** multiword lexemes are matched *before* single words, so a compound is never split.
- **306** single-word entries carry token type, source language, slang flag and gloss.
- Spelling variation is folded to a normal form, so *wa* and *wah* count as one item.
- Corpus yields **134** tokens over **76** distinct items.

Chef, drop me for Carrefour Obili.

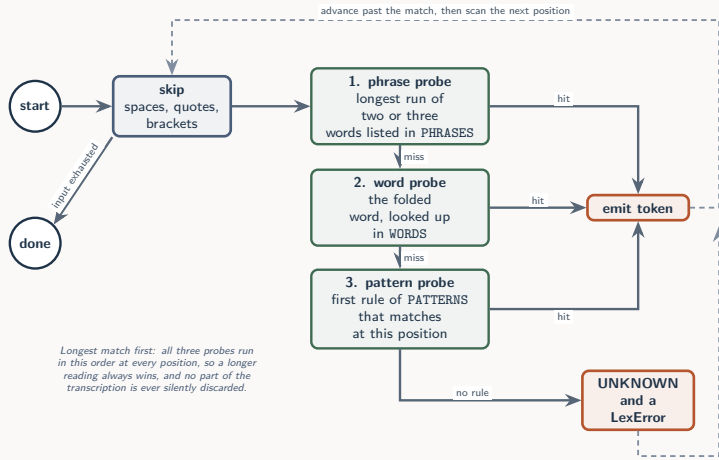
#	lexeme	token
1	Chef	VOC
2	,	SEP
3	drop	VERB
4	me	PRON
5	for	PREP
6	Carrefour	NOUN
7	Obili	NOUN
8	.	TERM

token stream

VOC SEP VERB PRON PREP NOUN NOUN TERM \$

The scanner as an automaton

Three probes, tried in this order at every position



The order *is* the specification:

phrase before word, word before pattern. That is what makes small small one adverb.

From a natural grammar to an LL(1) one

The grammar was written honestly first, then transformed

1. Remove left recursion

```
ClauseList → ClauseList SEP Clause
ClauseList → Clause
↓ immediate left recursion on ClauseList, introduced
ClauseList'
```

```
ClauseList → Clause ClauseList'
ClauseList' → SEP Clause ClauseList'
ClauseList' → ε
```

2. Left-factor the common prefix

```
Clause → Subj VP
Clause → Subj COP Items
↓ common prefix 'Subj', introduced Clause_f
```

```
Clause → Subj Clause_f
Clause_f → VP
Clause_f → COP Items
```

Both transformations are performed **in code**, not by hand: `remove_left_recursion` and `left_factor` run every time the analyzer starts, so the grammar shown and the grammar parsed cannot diverge.

The LL(1) table — and the one honest conflict

- FIRST and FOLLOW are computed to a fixed point, then the table is filled from them — nothing is transcribed by hand.
- **38** productions over **16** terminals.
- An empty cell is a parse error, and the parser says *which* terminal it expected.

Conflict at M[NGopt, NOUN]

After a noun phrase, a following NOUN could extend that phrase (Carrefour Obili as one name) or begin a new argument.

This is a genuine ambiguity — the dangling-else of this grammar. No rewriting removes it.

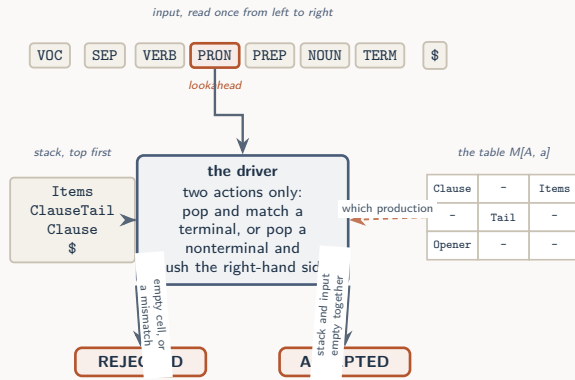
Resolved in favour of greedy compounding, which is the correct reading for every compound in the corpus.

0 undocumented conflicts. The one above is

declared in the source and checked by a unit test, so it cannot be quietly forgotten.

The parser as a stack machine

One token of lookahead, one stack, and a table that decides



Every step is recorded, so a rejection is justified with the exact stack, the lookahead and the table cell that was empty, rather than merely announced.

The driver contains no grammar. Change the grammar and the table changes; the code does not.

Results

Every acceptance carries a derivation; every rejection, a reason

Statements collected	15
Accepted	15
Rejected	0
Counter-examples	6
Correctly rejected	6
Tokens	134
Distinct items	76
Type/token ratio	56.7%

A rejection, with its reason

TRANSCRIPTION

Mbum, the flurble don wibble.

Accepted Multisend Bilingual Unknown word End structure

Or load a corpus statement NED — for Moby-Dick drop me.

REJECTED (lexical) 7 1 0

line 1, col 11: 'flurble' not covered by the lexical specification: line 1, col 23: 'wibble' not covered by the lexical specification

TOKENS CLAUSE ITEMS

Tokens Parse tree Parse trace Derivation FIRST / FOLLOW Corpus Statistics

Parse trace

Stack, remaining input and action at every step of the predictive parse.

STEP	STACK	REMAINING INPUT	ACTION
------	-------	-----------------	--------

FIRST and FOLLOW, computed at run time

Tokens Parse tree Parse trace **Derivation** FIRST / FOLLOW Corpus Statistics

Leftmost derivation

The same parse replayed as sentential forms.

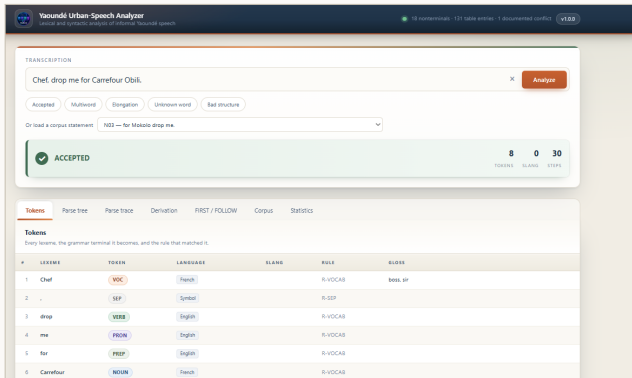
1	Statement
2	⇒ Preamble ClauseList TERM
3	⇒ Opener Preamble ClauseList TERM
4	⇒ VOC SepOpt Preamble ClauseList TERM
5	⇒ VOC SEP Preamble ClauseList TERM
6	⇒ VOC SEP ClauseList TERM
7	⇒ VOC SEP Clause ClauseList' TERM
8	⇒ VOC SEP Subj Clause_f ClauseList' TERM
9	⇒ VOC SEP Clause_f ClauseList' TERM
10	⇒ VOC SEP VP ClauseList' TERM
11	⇒ VOC SEP VERB Items ClauseList' TERM
12	⇒ VOC SEP VERB Item Items ClauseList' TERM
13	⇒ VOC SEP VERB NP Items ClauseList' TERM
14	⇒ VOC SEP VERB PRON Items ClauseList' TERM
15	⇒ VOC SEP VERB PRON Item Items ClauseList' TERM
16	⇒ VOC SEP VERB PRON PP Items ClauseList' TERM

The

parser reports the stack, the remaining input and the terminal it expected — not merely that the sentence failed.

The analyzer

One engine, three front ends, live on the web



Yaoundé Urban-Speech Analyzer
lexical and syntactic analysis of informal Yaoundé speech

10 nonterminals · 121 table entries · 1 documented conflict · v1.0.0

TRANSCRIPTION

Chef, drop me for Carrefour Obili.

Accepted Multword Stongation Unknown word Bad structure

Or load a corpus statement: N03 — for Mokele drop me.

✓ ACCEPTED

8 0 30
TOKENS SLANG STEPS

Tokens Parse tree Parse trace Derivation FIRST / FOLLOW Corpus Statistics

Tokens
Every lexeme, the grammar terminal it becomes, and the rule that matched it.

#	LEXEME	TOKEN	LANGUAGE	SLANG	RULE	GLOSS
1	Chef	VOC	French		R-VOCAB	boss, sir
2	,	SEP	Symbol		R-SEP	
3	drop	VERB	English		R-VOCAB	
4	me	PRON	English		R-VOCAB	
5	for	PREP	English		R-VOCAB	
6	Carrefour	NOUN	French		R-VOCAB	

- All three front ends construct the **same** Analyzer, so they cannot disagree about a verdict.
- Tabs for tokens, tree, trace, derivation, FIRST/FOLLOW, corpus, statistics — and **Grammar**, which proposes a spelling when a word is rejected.
- **No third-party packages** — the standard library only.
- Deployed over HTTPS:

<https://compiler-app.duckdns.org/>

What we found

The interesting result is where the method stops

What worked

- A restricted slice of the register is genuinely context-free and parses predictively.
- Multiword lexemes solve compounding at the **lexical** level, where it belongs.
- Normalising spelling variants makes frequency counts meaningful across transcriptions.

What did not

- Code-mixing operates **below the clause**, sometimes inside a single lexeme — a CFG cannot describe that.
- Aspect is marked by free particles, not inflection, so the lexer cannot resolve class from form.
- Noun compounding is **genuinely ambiguous**; we document the resolution rather than hide it.
- The corpus is transcribed by ear, without recordings.

Conclusion

- The lexical and syntactic phases of a compiler **do** apply to informal Yaoundé speech — over a deliberately restricted grammar, with 15 of 15 statements accepted and all 6 counter-examples rejected.
- Every number in this talk and in the report is **computed**, not asserted: the same code that analyses the corpus writes the document and these slides.
- The clearest next step is **more data**, not more grammar. Replacing the corpus regenerates every table, figure and percentage automatically.

Try it: <https://compiler-app.duckdns.org/>

```
python -m yca web · python -m yca test · python -m yca report
```

Thank you — questions?